

LLD – Analytics Module (OpenScholar)

1. Overview

This document defines the Low-Level Design (LLD) for the Analytics module of OpenScholar. The module is responsible for tracking user interactions (views, downloads) and providing aggregated metrics for performance-efficient querying.

The design follows a hybrid approach:

- Event-based logging (flexibility)
 - Aggregated counters (performance)
-

2. Scope

The Analytics module is responsible for:

- Track paper views
- Track paper downloads
- Store raw analytics events
- Maintain aggregated metrics per paper
- Provide analytics data for dashboards

Out of Scope:

- Advanced analytics (ranking, ML)
 - Real-time streaming analytics
-

3. Data Model (Relevant Tables)

Tables used:

- analytics_events
- paper_metrics
- papers
- users (optional)

analytics_events

- id (BIGSERIAL)
- paper_id (UUID)
- user_id (UUID, nullable)

- event_type (ENUM: view | download)
- created_at (TIMESTAMP)

paper_metrics

- paper_id (UUID, PK)
 - view_count (INT)
 - download_count (INT)
 - updated_at (TIMESTAMP)
-

4. API Contracts

4.1 Track View Event

Endpoint: POST /api/analytics/view

Request Body: { "paperId": "uuid" }

Response: { "message": "View recorded" }

4.2 Track Download Event

Endpoint: POST /api/analytics/download

Request Body: { "paperId": "uuid" }

Response: { "message": "Download recorded" }

4.3 Get Paper Metrics

Endpoint: GET /api/analytics/papers/{paperId}

Response: { "paperId": "uuid", "views": 120, "downloads": 45 }

5. Service Logic

5.1 trackEvent(paperId, userId, eventType)

Steps: 1. Validate paper exists and is approved 2. Insert new row into analytics_events 3. Update paper_metrics:

- Increment view_count or download_count

- Update timestamp
-

5.2 getPaperMetrics(paperId)

Steps: 1. Query paper_metrics table 2. Return counts

6. Aggregation Strategy

- Each event is stored in analytics_events (raw)
- paper_metrics maintains precomputed counts
- Updates are performed synchronously (MVP)

Future:

- Move aggregation to background worker (queue)
-

7. Validation Rules

- paper_id must exist
 - event_type must be valid enum
-

8. Performance Considerations

- Use index on analytics_events.paper_id
 - Use index on analytics_events.event_type
 - Avoid querying raw events for counts
 - Use aggregated table for reads
-

9. Security Considerations

- Prevent abuse (future: rate limiting)
 - Validate user identity if logged in
-

10. Edge Cases

- Duplicate rapid events (same user)
- Anonymous users (user_id null)
- High traffic spikes

11. Prisma Model Mapping

```
model AnalyticsEvent { id Int @id @default(autoincrement()) paperId String userId String? eventType String  
createdAt DateTime @default(now()) }
```

```
model PaperMetrics { paperId String @id viewCount Int @default(0) downloadCount Int @default(0)  
updatedAt DateTime }
```

12. Folder Structure

```
src/ app/api/analytics/view/route.ts app/api/analytics/download/route.ts app/api/analytics/papers/[id]/  
route.ts
```

```
modules/analytics/service.ts modules/analytics/repository.ts
```

13. Implementation Notes

- Use Prisma transactions for event + metric update
 - Keep writes lightweight
 - Avoid heavy joins
-

14. Acceptance Criteria

- System records views and downloads
 - Metrics are updated correctly
 - Metrics API returns accurate counts
 - System performs efficiently under load
-

15. Future Enhancements

- Redis caching for metrics
 - Queue-based async processing (Kafka/RabbitMQ)
 - User behavior analytics
 - Trending papers calculation
-

End of Document